

AlertZone

Système d'alerte citoyen géolocalisé

DOSSIER DE PROJET

Titre Professionnel — Concepteur Développeur d'Applications (CDA)

Niveau 6 (Bac+3/4)

Alain-Patrick GOMAS

IPSSI — Promotion NAT6-BD-27.1

Session 2027

Première version — novembre 2026

Sommaire

1. Résumé du projet (français)
2. Project summary (English)
3. Compétences du référentiel CDA couvertes
4. Expression des besoins — Cahier des charges fonctionnel
5. Gestion de projet et organisation
6. Spécifications fonctionnelles
7. Conception de la base de données (méthode MERISE)
8. Conception applicative (UML) et architecture multi-couches
9. Maquettes et interfaces utilisateur
10. Réalisation — extraits de code commentés
11. Sécurité (OWASP et RGPD)
12. Plan de tests et jeu d'essai
13. Déploiement — Docker et CI/CD
14. Veille technologique et sécurité
15. Situation de travail décrite en anglais
16. Conclusion et perspectives

1. Résumé du projet

AlertZone est une application web de sécurité citoyenne qui permet à un habitant de recevoir et de signaler, en temps quasi réel, les alertes qui surviennent dans sa zone géographique : agression, vol, phénomène météo dangereux, disparition de personne ou avis de recherche.

Le constat de départ est simple : lorsqu'un événement dangereux survient dans un quartier, l'information circule mal. Les personnes les plus exposées — celles qui se trouvent sur place — sont souvent les dernières informées. Les canaux existants sont soit imprécis géographiquement (alertes nationales), soit peu fiables (réseaux sociaux non vérifiés). AlertZone répond à une question concrète : *comment diffuser une alerte fiable aux seules personnes présentes dans la zone concernée ?*

L'application repose sur une **carte interactive** (Leaflet / OpenStreetMap) où chaque alerte est matérialisée par un marqueur coloré selon son type, entouré d'un rayon d'impact. Un citoyen peut filtrer les alertes, consulter leur détail, en signaler une nouvelle, la confirmer ou la marquer comme fausse. Une autorité valide les signalements et publie les alertes officielles ; le système enrichit automatiquement la carte avec les vigilances météo issues d'une API externe (OpenWeatherMap).

Techniquement, le projet est construit selon une architecture **multi-couches découplée** : une API REST développée avec **Symfony 7 (PHP 8.2)**, une base de données relationnelle **MySQL 8** pilotée par l'ORM Doctrine, et un front-end **React 18** (Vite) qui consomme l'API en JSON. L'ensemble est conteneurisé avec **Docker** et versionné sur **GitHub**. Le projet mobilise les trois activités du référentiel CDA : développer une application sécurisée, concevoir une application organisée en couches avec sa base de données, et préparer son déploiement dans une démarche DevOps.

Innovation apportée : un *algorithme de fiabilité* qui pondère chaque alerte selon la validation communautaire (confirmations contre faux signalements) et son ancienneté, afin de faire remonter l'information vérifiée et d'estomper les fausses alertes — réponse directe au principal défaut des plateformes de signalement citoyen : la désinformation.

2. Project summary (English)

AlertZone *is a citizen-safety web application that lets residents receive and report, in near real time, the incidents happening in their geographic area: assault, theft, dangerous weather, missing persons or police wanted notices.*

The problem it solves is straightforward: when a dangerous event occurs in a neighbourhood, information spreads slowly and unreliably. The people most at risk are often the last to know. Existing channels are either geographically too broad (national alerts) or untrustworthy (unverified social media). AlertZone answers one concrete question: how can a reliable alert be delivered only to the people actually located in the affected area?

The application is built around an ***interactive map*** *(Leaflet / OpenStreetMap). Each alert is shown as a coloured marker with an impact radius. A citizen can filter alerts, open their details, submit a new one, confirm an alert or flag it as false. An authority validates citizen reports and issues official alerts, while the system automatically adds weather warnings pulled from an external API (OpenWeatherMap).*

From a technical standpoint, the project follows a ***decoupled multi-layer architecture****: a REST API built with* ***Symfony 7 (PHP 8.2)****, a relational* ***MySQL 8*** *database accessed through the Doctrine ORM, and a* ***React 18*** *front-end that consumes the API as JSON. Everything is containerised with* ***Docker*** *and version-controlled on* ***GitHub****. The project covers the three activity blocks of the CDA qualification: developing a secure application, designing a layered application with its database, and preparing its deployment following a DevOps approach.*

Key innovation: *a reliability scoring algorithm that weights each alert according to community validation (confirmations versus false reports) and its age, so that verified information rises to the top while false alarms fade out — a direct answer to the main weakness of citizen-reporting platforms: misinformation.*

3. Compétences du référentiel CDA couvertes

Le tableau ci-dessous relie chaque compétence du référentiel du titre professionnel *Concepteur Développeur d'Applications* aux réalisations du projet AlertZone et à la section du dossier qui en apporte la preuve.

Activité-type 1 — Développer une application sécurisée

Compétence	Mise en œuvre dans AlertZone	Section
Installer et configurer son environnement de travail	VS Code, Node/Vite, Composer, Docker Desktop ; dépôt Git structuré (main / develop / feature).	5, 13
Développer des interfaces utilisateur	Front React 18 : carte Leaflet, filtres, liste synchronisée, modales de détail et de signalement, responsive (mobile/desktop).	9, 10
Développer des composants métier	Services Symfony (AlerteService, WeatherService, NotificationService) ; logique de création et de validation des alertes.	8, 10
Contribuer à la gestion d'un projet informatique	Méthode Kanban solo, planning de rattrapage, suivi GitHub Projects, jalons de la formation.	5

Activité-type 2 — Concevoir et développer une application organisée en couches

Compétence	Mise en œuvre dans AlertZone	Section
Analyser les besoins et maquetter une application	Cahier des charges fonctionnel, personas, zoning et maquettes des écrans clés.	4, 9
Définir l'architecture logicielle	Architecture API REST + SPA, pattern MVC, 3-tiers, séparation des responsabilités (SOLID / PSR-12).	8
Concevoir et mettre en place une base de données relationnelle	Modélisation MERISE complète (MCD, MLD, MPD), normalisation 3FN, MySQL 8 / Doctrine.	7, 10
Développer des composants d'accès aux données	Entités et Repositories Doctrine, requêtes paramétrées, endpoints REST CRUD.	10

Activité-type 3 — Préparer le déploiement d'une application sécurisée

Compétence	Mise en œuvre dans AlertZone	Section
Préparer et exécuter les plans de tests	Tests unitaires PHPUnit (logique métier), tests fonctionnels d'API (WebTestCase), tests front Vitest.	12

Préparer et documenter le déploiement	Dockerfile + docker-compose (front / api / db), README d'installation, variables d'environnement.	13
Contribuer à la mise en production (DevOps)	Pipeline CI/CD GitHub Actions : installation, lint, tests automatisés à chaque push.	13

Note d'avancement (première version). La phase de conception (sections 4 à 9) est terminée et documentée. Le développement est engagé : l'API Symfony, les entités Doctrine et le front React connecté sont réalisés. Les briques restantes — authentification JWT complète, intégration météo automatisée, tests et CI — sont conçues et planifiées pour la version finale (février 2027).

4. Expression des besoins — Cahier des charges fonctionnel

4.1 Contexte métier

Le domaine concerné est celui de la **sécurité publique de proximité** et de la gestion d'alertes citoyennes. Deux grands profils d'utilisateurs sont visés : les **habitants** d'une ville ou d'un quartier qui souhaitent être prévenus des dangers locaux, et les **autorités locales** (police municipale, mairie, pompiers) qui ont besoin de diffuser rapidement une information vérifiée à une population précise.

4.2 Objectifs (SMART)

1. Permettre à tout citoyen inscrit de **signaler un incident géolocalisé** en moins de 60 secondes (type, titre, position, gravité).
2. **Diffuser une alerte** aux utilisateurs abonnés à la zone concernée dès sa validation, par notification in-app.
3. Offrir une **carte interactive filtrable** affichant les incidents actifs et leur rayon d'impact.
4. **Intégrer automatiquement** les alertes météo d'une zone via une API externe.
5. Donner aux autorités un **outil de modération** et d'émission d'avis de recherche.
6. **Garantir la sécurité des données** (mots de passe hachés, RGPD, OWASP) avec zéro faille critique.

4.3 Personas (synthèse)

Persona	Profil	Besoin principal
Camille, 32 ans	Habitante, parent	Être prévenue vite d'un danger près de l'école de ses enfants.
Officier Martin	Police municipale	Diffuser un avis de recherche ciblé et modérer les signalements.
Admin technique	Administrateur	Superviser la plateforme, gérer utilisateurs et types d'alerte.

4.4 Exigences techniques (socle imposé)

- Architecture : **API Symfony + front React séparé**.
- Composants : PHP 8.2, Symfony 7, React 18, MySQL 8, Docker.
- Contraintes imposées : Docker, CI/CD, tests automatisés, sécurité OWASP, **au moins une API externe**.
- Qualité : code maintenable, conventions PSR-12, séparation des responsabilités, documentation.

4.5 Risques et critères de succès

Risque	Probabilité	Mitigation
Délai court pour tout implémenter	Élevée	MVP priorisé (F1–F5), fonctions secondaires en option.

Dépendance à l'API météo externe	Moyenne	Gestion d'erreur + repli (mock) si l'API ne répond pas.
Quota gratuit OpenWeatherMap dépassé	Faible	Mise en cache des appels ; volumes faibles en démonstration.

Critères de succès : F1 à F5 + F8/F9 implémentées et testées ; carte fonctionnelle ; API externe opérationnelle ; zéro faille OWASP critique ; application déployable via `docker-compose up`.

5. Gestion de projet et organisation

5.1 Méthode

Le projet étant mené en solo avec un délai contraint, j'ai retenu une approche **Agile Kanban** plutôt qu'un Scrum complet : un flux continu de tâches (À faire / En cours / Terminé) sans cérémonies lourdes, suivi sur un tableau **GitHub Projects**, accompagné d'un journal de bord hebdomadaire tenant lieu de revue de sprint.

5.2 Planning

Phase	Contenu	État
Cadrage	Choix du sujet, CDCF	Terminé
Conception	UI/UX, MERISE, UML, architecture	Terminé
Développement	Front React, API Symfony, entités, connexion	En cours
Intégration	API OpenWeatherMap, score de fiabilité	Planifié
Qualité	Sécurité OWASP, tests, CI/CD	Planifié
Livraison	Dockerisation finale, dossier, soutenance	Planifié

5.3 Versioning (Git)

Stratégie **GitFlow simplifié** : `main` (versions stables, tags `v0.x` puis `v1.0`), `develop` (intégration continue) et `feature/*` (une branche par fonctionnalité, fusionnée via Pull Request après test). Commits fréquents et explicites selon la convention `feat:` / `fix:` / `docs:` / `test:`. Les fichiers de configuration (Docker, CI) sont versionnés, un `README.md` documente l'installation.

6. Spécifications fonctionnelles

Le périmètre fonctionnel est découpé en onze fonctionnalités (F1 à F11). Les fonctionnalités cœur (MVP) sont F1 à F5, F8 et F9.

Réf.	Fonctionnalité	Description
F1	Comptes et rôles	Inscription, connexion, profil. Trois rôles : CITOYEN, AUTORITE, ADMIN.
F2	Carte interactive	Alertes actives sur carte Leaflet, marqueurs colorés par type, rayon d'impact.
F3	Signalement	Création d'une alerte : type, titre, description, position, gravité. Statut initial : en attente.
F4	Filtrage et recherche	Filtrer par type, statut, proximité ; recherche textuelle.
F5	Réactions citoyennes	Confirmer, signaler une fausse alerte, commenter — fiabilise l'information.
F6	Abonnement aux zones	S'abonner à des zones pour en recevoir les alertes.
F7	Notifications	Réception in-app des alertes des zones abonnées, marquage lu/non-lu.
F8	Modération (Autorité)	Valider/rejeter les signalements, émettre des alertes officielles.
F9	Avis de recherche (Autorité)	Personne disparue / avis de recherche avec photo, identité, description.
F10	Alertes météo (Système)	Interrogation d'OpenWeatherMap et création automatique d'alertes de vigilance.
F11	Administration (Admin)	Gestion des utilisateurs, supervision des alertes, gestion des zones et types.

Diagramme de cas d'utilisation

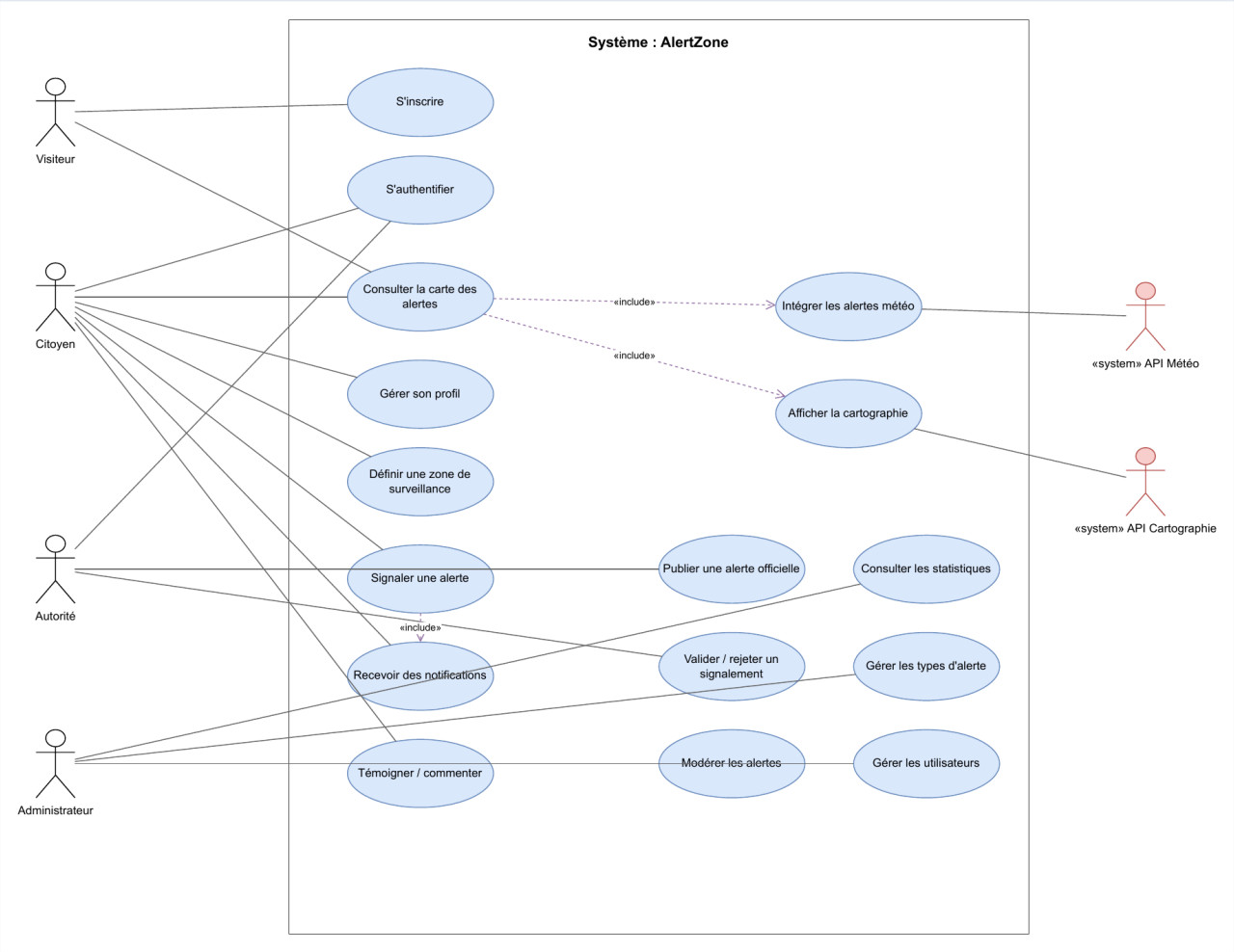


Figure 1 — Cas d'utilisation : acteurs (Visiteur, Citoyen, Autorité, Administrateur) et systèmes externes.

7. Conception de la base de données (méthode MERISE)

La modélisation suit les trois étapes de MERISE : **MCD** (conceptuel, indépendant de la technique), **MLD** (logique / relationnel) puis **MPD** (physique, propre à MySQL). Cette progression garantit une base normalisée en 3e forme normale.

7.1 Dictionnaire des données (extrait)

Donnée	Entité	Type	Description
email	UTILISATEUR	VARCHAR unique	Identifiant de connexion
mot_de_passe	UTILISATEUR	VARCHAR(255)	Haché (bcrypt/argon2), jamais en clair
latitude / longitude	ALERTE	DECIMAL(10,7)	Coordonnées géographiques
niveau_gravite	ALERTE	TINYINT (1–5)	Échelle de gravité
statut	ALERTE	ENUM	EN_ATTENTE / VALIDEE / REJETEE / EXPIREE
rayon	ALERTE	INT (mètres)	Rayon d'impact

7.2 Modèle Conceptuel de Données (MCD)

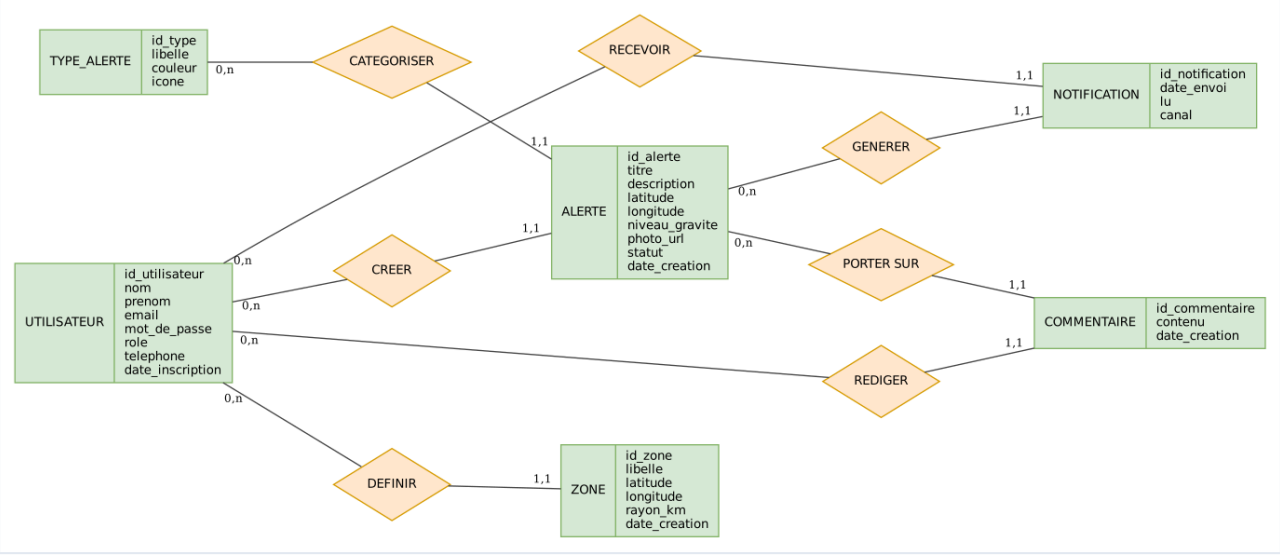


Figure 2 — MCD : entités et associations avec cardinalités MERISE (0,n / 1,1).

Lecture des cardinalités : un UTILISATEUR possède 1,1 rôle ; une ALERTE est émise par 1,1 utilisateur et classée dans 1,1 type ; NOTIFICATION, COMMENTAIRE et ABONNEMENT sont des entités associatives matérialisant des relations plusieurs-à-plusieurs porteuses d'attributs.

7.3 Modèle Logique de Données (MLD)

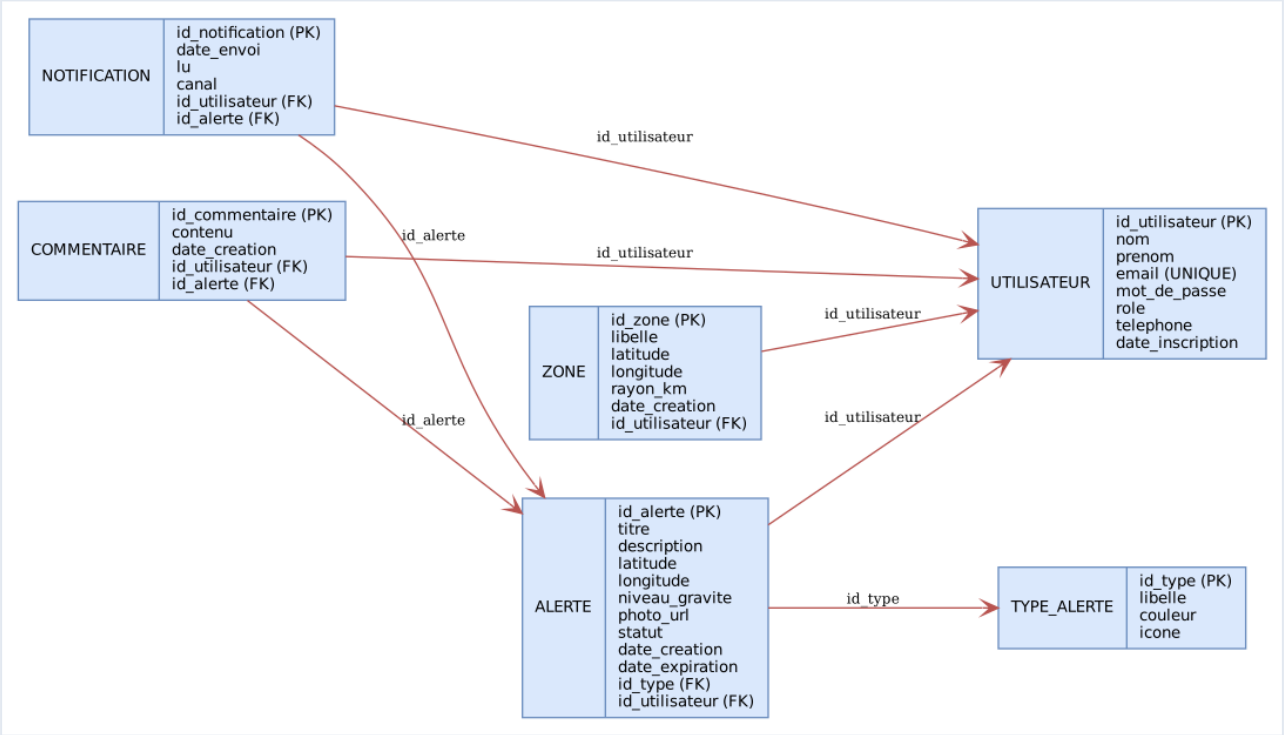


Figure 3 — MLD : passage aux tables avec clés primaires (PK) et étrangères (FK).

7.4 Modèle Physique de Données (MPD)

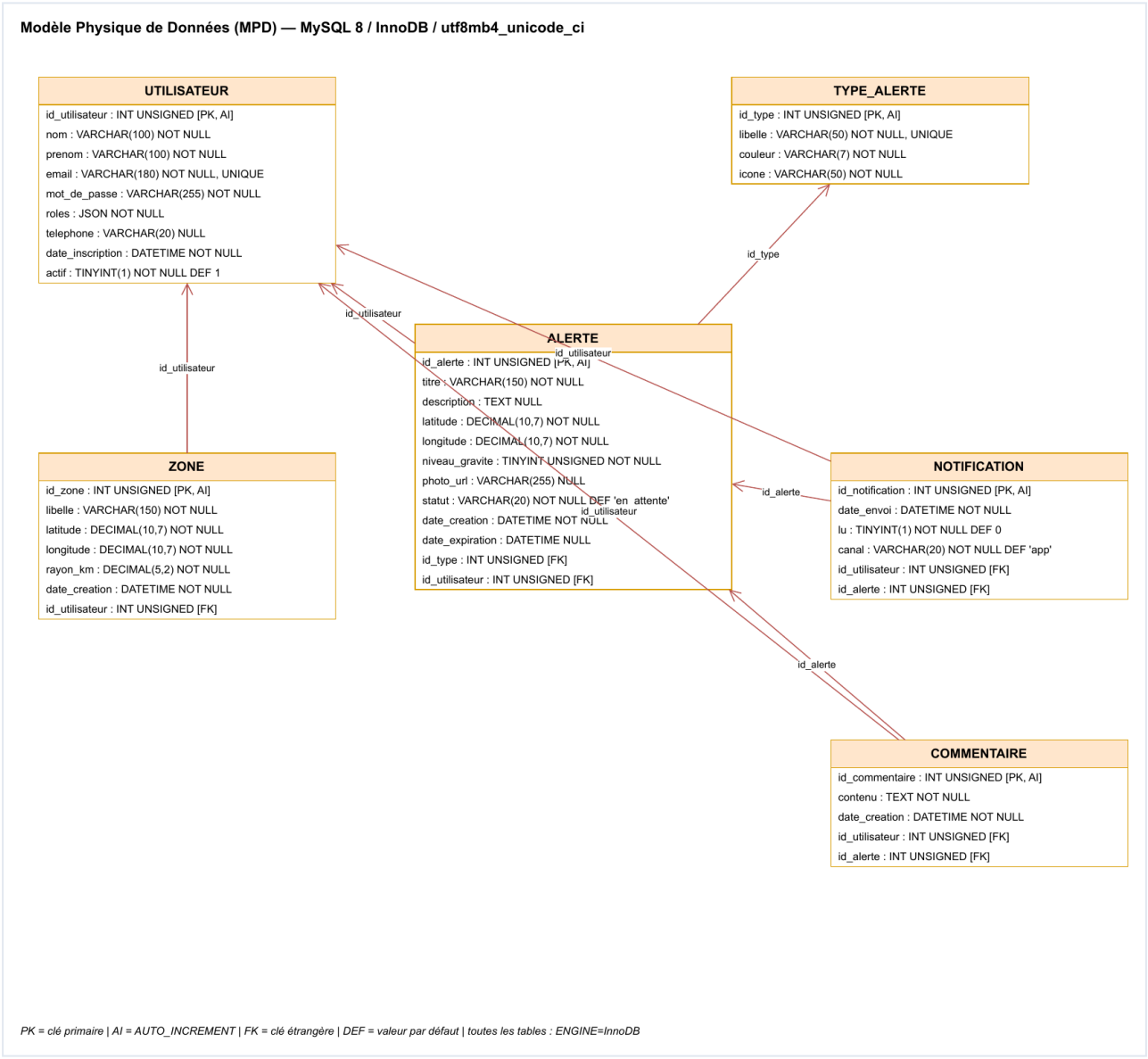


Figure 3 bis — MPD : tables MySQL 8 / InnoDB avec types, clés et contraintes.

Extrait du script de création (SQL) correspondant :

```
CREATE TABLE utilisateur (
    id_utilisateur   INT UNSIGNED NOT NULL AUTO_INCREMENT,
    nom              VARCHAR(100) NOT NULL,
    email            VARCHAR(180) NOT NULL,
    mot_de_passe     VARCHAR(255) NOT NULL,   -- hash argon2, jamais en clair
    roles            JSON NOT NULL,
    date_inscription DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_utilisateur),
    UNIQUE KEY uq_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE alerte (
    id_alerte       INT UNSIGNED NOT NULL AUTO_INCREMENT,
    titre           VARCHAR(150) NOT NULL,
    description     TEXT,
    latitude        DECIMAL(10,7) NOT NULL,
    longitude       DECIMAL(10,7) NOT NULL,
    niveau_gravite  TINYINT UNSIGNED NOT NULL DEFAULT 1,
    statut          VARCHAR(20) NOT NULL DEFAULT 'validee',
    id_type         INT UNSIGNED NOT NULL,
    id_utilisateur  INT UNSIGNED NOT NULL,
    PRIMARY KEY (id_alerte),
    CONSTRAINT fk_alerte_type FOREIGN KEY (id_type) REFERENCES type_alerte(id_type),
    CONSTRAINT fk_alerte_user FOREIGN KEY (id_utilisateur) REFERENCES
utilisateur(id_utilisateur),
    INDEX idx_statut (statut),
    INDEX idx_geo (latitude, longitude)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Justification : modèle en 3e forme normale, aucune redondance. Les types et rôles sont externalisés en tables de référence. Moteur InnoDB (clés étrangères, transactions), encodage utf8mb4 (accents). Index sur les colonnes filtrées (`statut`, position GPS) et `ON DELETE CASCADE` prévu pour la conformité RGPD (suppression du compte et de ses données).

8. Conception applicative (UML) et architecture

8.1 Diagramme de classes

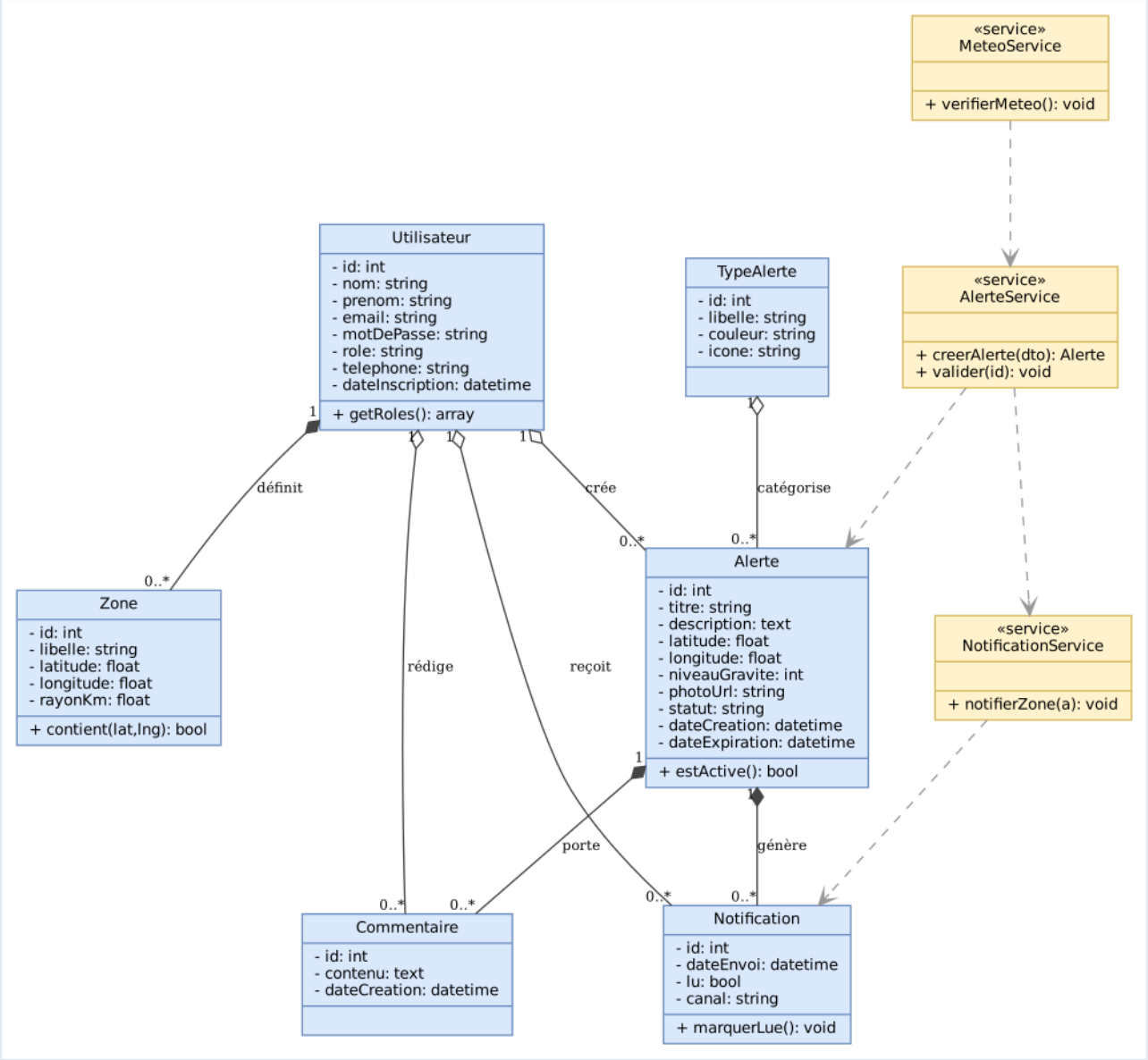


Figure 4 — Diagramme de classes : entités métier + couche de services.

La **composition** (losange plein) relie Alerte et Notification : une notification n'existe pas sans son alerte. L'**agrégation** (losange vide) relie Utilisateur et Alerte : les alertes existent indépendamment de leur auteur.

8.2 Diagramme de séquence — « Signaler une alerte »

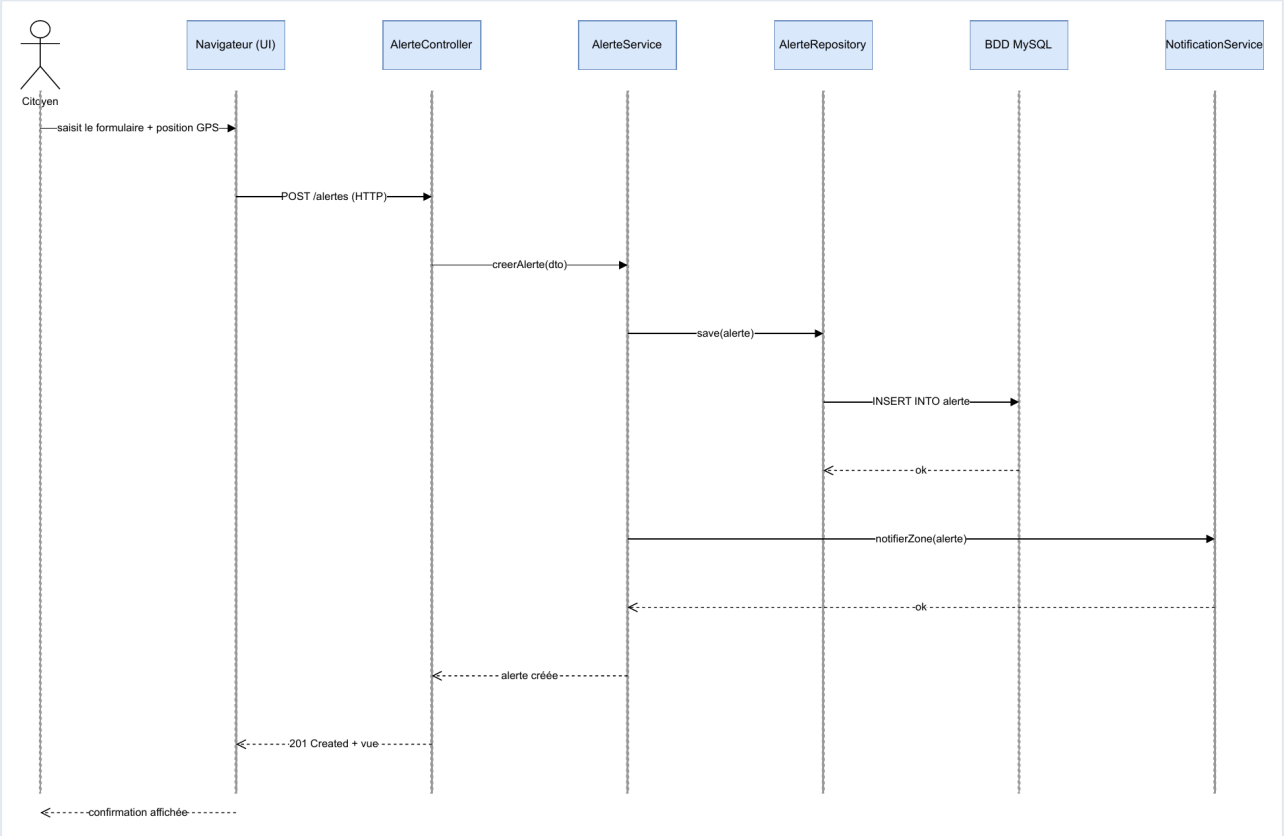


Figure 5 — Flux : Front → Controller → Service → Repository → BDD, puis notification.

8.3 Diagramme de séquence — « Intégration météo (API externe) »

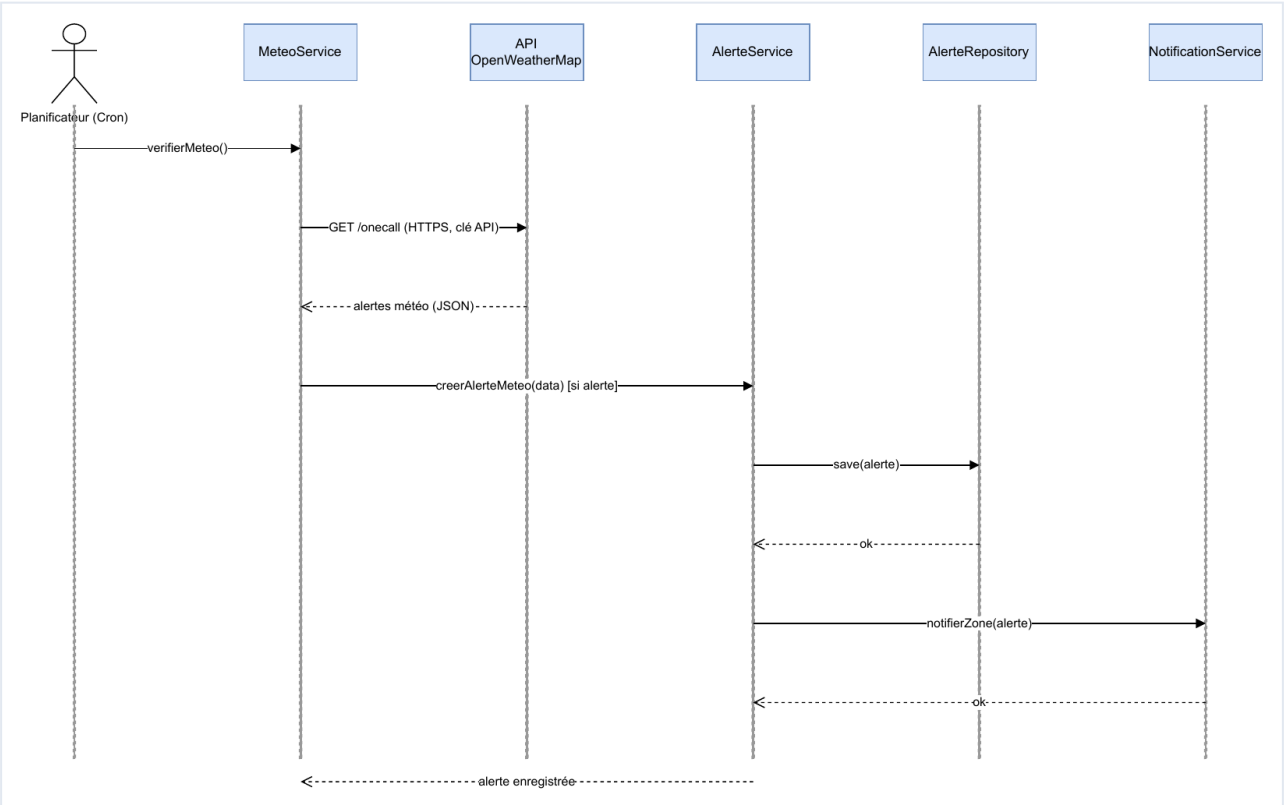


Figure 6 — Le planificateur interroge OpenWeatherMap et crée automatiquement l'alerte.

8.4 Architecture multi-couches

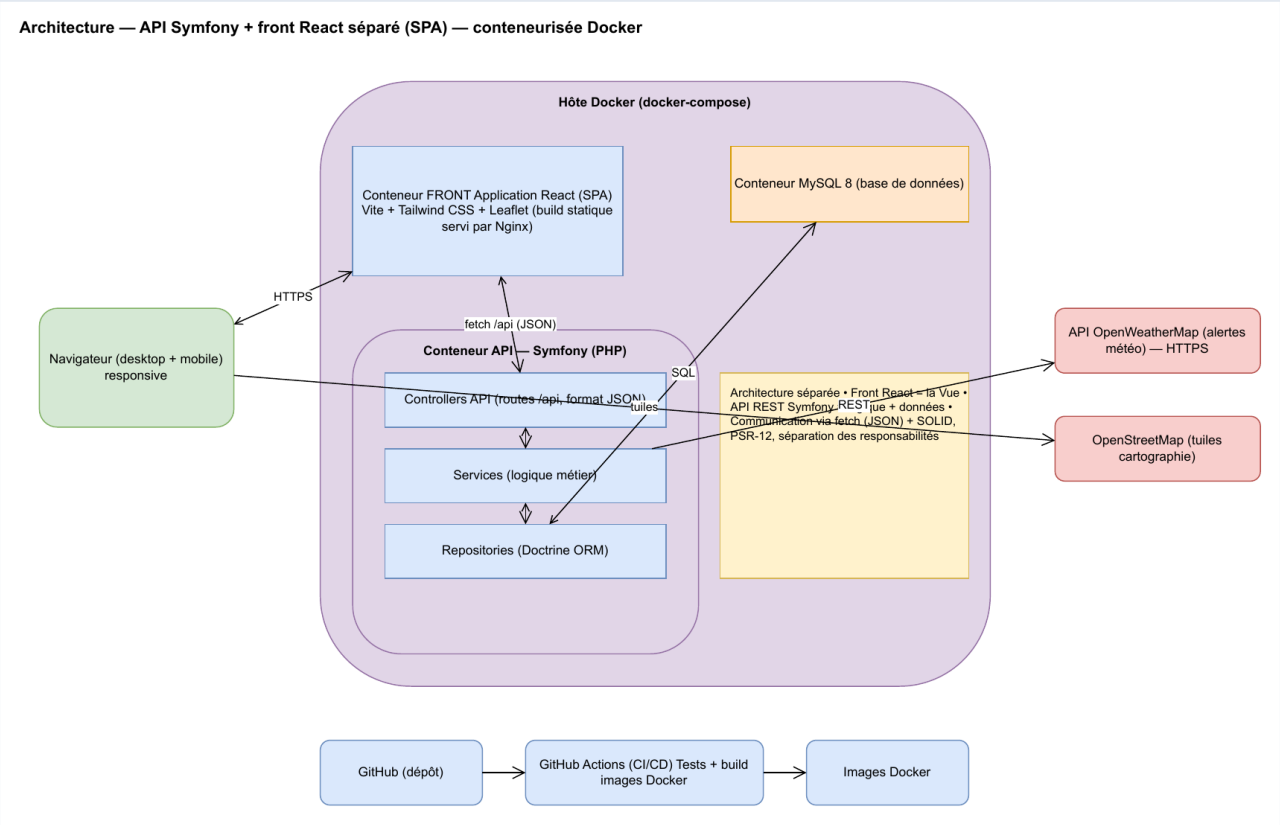


Figure 7 — Architecture : front React (SPA) + API Symfony + MySQL, conteneurisée Docker.

Pattern MVC (couche logique) : Controllers (reçoivent les requêtes HTTP, renvoient du JSON), Services (logique métier, aucune règle dans le contrôleur), Modèles (entités + repositories Doctrine), Vue (le front React). **Architecture 3-tiers (tiers physique) :** client (navigateur), serveur d'application (Symfony), serveur de données (MySQL) — des conteneurs séparés orchestrés par Docker Compose. La distinction couche logique / tiers physique est volontairement maintenue : les trois couches peuvent tourner sur un même hôte.

9. Maquettes et interfaces utilisateur

9.1 Zoning de l'écran principal



9.2 Charte graphique

Élément	Valeur	Usage
Bleu principal	#1E3A8A	Header, boutons primaires, identité
Rouge alerte	#DC2626	Urgences, agressions, actions critiques
Ambre	#F59E0B	Météo / vigilance
Violet	#7C3AED	Personne disparue / avis de recherche
Police	Inter	Titres (gras) et texte courant

Approche **Mobile First**, retours visuels clairs (toasts), accessibilité (contrastes AA). Le geste central — « signaler » — est accessible en un clic via un bouton flottant depuis n'importe quel écran.

> Les captures d'écran de l'application fonctionnelle (carte, liste, détail, formulaire de signalement) seront insérées en annexe de la version finale, une fois l'interface figée.

10. Réalisation — extraits de code commentés

Cette section présente des extraits **significatifs** du code produit, un par couche, pour illustrer la mise en œuvre concrète de l'architecture.

10.1 Couche données — Entité Doctrine (API Symfony)

L'entité `Alerte` traduit la table `alerte` du MPD en classe PHP. Les attributs Doctrine (`#[ORM\...]`) décrivent le mapping objet-relationnel.

```
#[ORM\Entity(repositoryClass: AlerteRepository::class)]
#[ORM\Table(name: 'alerte')]
class Alerte
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 150)]
    private string $titre = '';

    #[ORM\Column(type: 'decimal', precision: 10, scale: 7)]
    private string $latitude = '0';

    #[ORM\Column]
    private int $niveauGravite = 1;

    #[ORM\ManyToOne(targetEntity: TypeAlerte::class)]
    #[ORM\JoinColumn(name: 'id_type', nullable: false)]
    private ?TypeAlerte $type = null;
    // ... getters / setters
}
```

10.2 Couche métier + présentation API — Contrôleur REST

Le contrôleur expose les endpoints. Il délègue la logique et renvoie du JSON ; aucune règle métier n'y figure (séparation des responsabilités).

```
#[Route('/api')]
class AlerteController
{
    #[Route('/alertes', methods: ['GET'])]
    public function liste(AlerteRepository $repo): JsonResponse
    {
        $data = array_map([$this, 'serialiser'],
            $repo->findBy([], ['dateCreation' => 'DESC']));
        return new JsonResponse($data);
    }

    #[Route('/alertes', methods: ['POST'])]
    public function creer(Request $req, EntityManagerInterface $em,
                          TypeAlerteRepository $types): JsonResponse
    {
        $d = json_decode($req->getContent(), true) ?? [];
        if (empty($d['titre']) || empty($d['type'])) {
            return new JsonResponse(['erreur' => 'Titre et type obligatoires.'], 400);
        }
        $a = (new Alerte())
            ->setTitre($d['titre'])
            ->setType($types->findOneBy(['cle' => $d['type']]))
            ->setNiveauGravite((int) ($d['niveauGravite'] ?? 1));
        $em->persist($a);
        $em->flush();
        return new JsonResponse($this->serialiser($a), 201);
    }
}
```

10.3 Couche présentation — Service d'appel API (React)

Côté front, un service centralise les appels HTTP. Les données remplacent les données fictives du prototype.

```
const BASE = import.meta.env.VITE_API_URL || 'http://localhost:8000/api'

export async function chargerAlertes() {
  const res = await fetch(`${BASE}/alertes`)
  if (!res.ok) throw new Error('API indisponible')
  return res.json()
}

export async function creerAlerte(donnees) {
  const res = await fetch(`${BASE}/alertes`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(donnees),
  })
  if (!res.ok) throw new Error('Création impossible')
  return res.json()
}
```

10.4 Interface — composant React connecté

Le composant racine charge les alertes depuis l'API au démarrage, avec repli sur les données locales si l'API est indisponible (robustesse de la démonstration).

```
useEffect(() => {
  api.chargerAlertes()
    .then((data) => { setAlertes(data); setConnecte(true) })
    .catch(() => { setConnecte(false) })  // repli mode maquette
}, [])
```

11. Sécurité (OWASP et RGPD)

Risque OWASP	Mesure dans AlertZone
Injection SQL	Doctrine ORM + requêtes paramétrées exclusivement ; aucune concaténation d'entrée utilisateur.
XSS	React échappe par défaut le contenu rendu ; validation/échappement des entrées côté API.
CSRF	Authentification API par **JWT** + vérification de l'origine (CORS restreint en production).
Authentification	Mots de passe hachés en **bcrypt/argon2** via le hasher Symfony ; jamais stockés en clair.
Force brute	Limitation des tentatives via le composant **Rate Limiter** de Symfony.
Contrôle d'accès	Rôles Symfony (ROLE_CITOYEN, ROLE_AUTORITE, ROLE_ADMIN) ; un citoyen n'accède pas aux écrans de modération.
Validation des entrées	Contraintes de validation Symfony sur les entités (formats, longueurs, types).

RGPD : l'utilisateur peut supprimer son compte et ses données (suppression en cascade au niveau BDD) ; politique de confidentialité intégrée ; données de démonstration uniquement ; HTTPS en production ; secrets (clé API, mot de passe BD, secret JWT) injectés par variables d'environnement, jamais commités.

12. Plan de tests et jeu d'essai

12.1 Tests unitaires (PHPUnit)

Sur la logique métier : `AlerteService::creerAlerte()` renvoie bien une alerte au statut attendu ; calcul de la distance entre deux points GPS pour déterminer l'appartenance à une zone.

12.2 Tests fonctionnels d'API (WebTestCase)

Cas de test	Entrée	Résultat attendu
Lister les alertes	GET /api/alertes	200 + tableau JSON
Créer une alerte valide	POST /api/alertes (titre + type)	201 + alerte créée
Créer sans titre	POST /api/alertes (type seul)	400 + message d'erreur
Confirmer une alerte	PATCH /api/alertes/1/confirmer	200 + compteur +1

12.3 Tests front (Vitest + React Testing Library)

Le filtre masque bien les marqueurs du type décoché ; le compteur du header reflète le nombre d'alertes visibles.

12.4 Autres tests

Test d'intégration de l'appel réel à OpenWeatherMap avec gestion d'erreur ; test de performance : chargement de la carte avec 100 alertes en moins de 2 s. Les tests sont exécutés en local et dans la CI GitHub Actions.

13. Déploiement — Docker et CI/CD

13.1 Conteneurisation

Un fichier `docker-compose.yml` orchestre les services : `api` (PHP/Symfony), `db` (MySQL avec volume persistant) et, en production, `front` (Nginx servant le build React). Lancement en une commande :

```
$ docker compose up --build
```

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: alertezone
    volumes: [ "db_data:/var/lib/mysql" ]
  api:
    build: ./backend
    ports: [ "8000:8000" ]
    depends_on: { db: { condition: service_healthy } }
    environment:
      DATABASE_URL: "mysql://alerte:alerte@db:3306/alertezone"
volumes: { db_data: {} }
```

13.2 Environnements

`APP_ENV=dev` (debug activé) en local, `APP_ENV=prod` (cache optimisé) en production. Les secrets sont injectés par variables d'environnement.

13.3 CI/CD (GitHub Actions)

À chaque push sur `develop` : installation des dépendances (Composer + npm), lint (PHP-CS-Fixer / ESLint), exécution des tests (PHPUnit + Vitest). Sur un tag `main` : build de l'image Docker. Une procédure de déploiement manuel documentée est prévue en contexte pédagogique.

14. Veille technologique et sécurité

La veille est une compétence continue du métier de concepteur développeur. J'ai organisé la mienne autour de trois axes et de sources identifiées.

Axe	Sources suivies	Application au projet
Framework back-end	symfony.com/blog, release notes Symfony, Doctrine docs	Choix de Symfony 7 (LTS), bonnes pratiques ORM, attributs PHP 8.
Front-end	react.dev, documentation Vite, newsletter « This Week in React »	Hooks (useState, useEffect, useMemo), découplage SPA / API.
Sécurité	OWASP Top 10, bulletins ANSSI (CERT-FR), Snyk	Cartographie des risques (section 11), hachage, validation, CORS.

Méthode : agrégation des flux RSS dans un lecteur, synthèse mensuelle dans mon journal de bord, et application directe au code (ex. : la lecture de l'OWASP Top 10 2021 a orienté la stratégie d'authentification par JWT et la limitation de débit). Cette veille sécurité est particulièrement critique pour AlertZone, qui manipule des données personnelles (email, localisation).

15. Situation de travail décrite en anglais

Description, en anglais, d'une situation de travail ayant nécessité une recherche d'information technique en langue anglaise (exigence du référentiel CDA).

> ***Integrating the OpenWeatherMap external API***
>
> *During the development of AlertZone, I had to integrate an external weather API so that the system could automatically create alerts when a weather warning is issued for a given area. I chose the **OpenWeatherMap API** because it offers a free tier and relays official national weather-agency warnings.*
>
> *The official documentation is written in English only. I had to read the One Call API 3.0 reference to understand how to authenticate my requests with an API key, which endpoint returns weather alerts, and how the JSON response is structured — in particular the `alerts` array, which contains the `event`, `start`, `end` and `description` fields.*
>
> *A specific difficulty was handling the case where the API does not respond or returns an error (rate limit exceeded, invalid key). On a technical forum (Stack Overflow), I found a recommended pattern: wrap the external call in a try/catch block, log the failure, and fall back gracefully instead of breaking the whole feature. I applied this pattern in my `WeatherService`, so that a temporary failure of the external API never prevents the rest of the application from working.*
>
> *This situation required me to read technical English accurately, compare several sources (official docs and community answers), and translate that understanding into working, resilient code. It confirmed that being able to read English documentation is an everyday skill for a developer, since most up-to-date technical references are published in English first.*

16. Conclusion et perspectives

Le projet AlertZone m'a permis de dérouler l'ensemble du cycle de développement d'une application sécurisée organisée en couches : de l'**expression du besoin** jusqu'à la **conception** (MERISE, UML, architecture), puis le **développement** d'une API Symfony et d'un front React connectés, et la préparation du **déploiement** conteneurisé.

Au-delà de la technique, ce projet m'a appris à **piloter un projet en autonomie** : prioriser un MVP sous contrainte de temps, documenter mes choix et savoir les justifier. Le changement de sujet en cours de route a été transformé en atout : il m'a obligé à affiner ma méthode de cadrage.

Perspectives d'évolution

- Finaliser l'**authentification JWT** complète et les trois rôles.
- Mettre en service l'**intégration météo automatique** (OpenWeatherMap) et le **score de fiabilité** des alertes (innovation).
- Compléter la **couverture de tests** et finaliser le **pipeline CI/CD**.
- À plus long terme : notifications push, application mobile consommant la même API, et partenariat avec les systèmes officiels pour les avis de recherche.

Cette première version du dossier témoigne d'une conception aboutie et d'un développement engagé ; la version finale présentera l'application complète, dockerisée et déployée, accompagnée de son dépôt GitHub.